ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Интеграция общего (системного) и специального программного обеспечения в программных комплексах.

по курсу: Основы программной инженерии

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4434		Р. А. Кочетков
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ЗАДАНИЕ 5

ВВЕДЕНИЕ..... 6

1 Лабораторная работа №3 — реализация и интеграция..... **Error! Bookmark not defined.**

1.1 Запуск ..... **Error! Bookmark not defined.**

1.2 Структура и поведение ..... **Error! Bookmark not defined.**

1.3 Локальная разработка без Docker.... **Error! Bookmark not defined.**

1.4 Демонстрация ..... **Error! Bookmark not defined.**

1.5 Проверки и эксплуатация..... **Error! Bookmark not defined.**

1.6 Роли и выводы ..... **Error! Bookmark not defined.**

2 Контракт API ..... **Error! Bookmark not defined.**

3 Проектирование и эксплуатация базы данных**Error! Bookmark not defined.**

3.1 Состав таблиц и связи..... **Error! Bookmark not defined.**

3.2 Роли и доступ ..... **Error! Bookmark not defined.**

3.3 Индексы для порядка 100 тысяч объектов**Error! Bookmark not defined.**

3.3.1 Оценка ёмкости ..... **Error! Bookmark not defined.**

3.4 Транзакции и ограничения..... **Error! Bookmark not defined.**

3.5 Резервное копирование и восстановление**Error! Bookmark not defined.**

4 Проверка ЛР №3..... **Error! Bookmark not defined.**

Серверные интеграционные тесты..... **Error! Bookmark not defined.**

Проверки клиента..... **Error! Bookmark not defined.**

Нагрузочный набор и измерения..... **Error! Bookmark not defined.**

Системная интеграция ..... **Error! Bookmark not defined.**

Автоматический отдельный тестовый стенд**Error! Bookmark not defined.**

ЗАКЛЮЧЕНИЕ 13

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 14

ПРИЛОЖЕНИЕ А Результаты проверки интерфейса**Error! Bookmark not defined.**

ЗАДАНИЕ

Проектирование, оптимизация и эксплуатация реляционной базы данных PostgreSQL: состав таблиц, ролевой доступ, индексы под нагрузку 100 тыс. объектов, транзакции, триггеры целостности и резервное копирование.

Задание выдал: _____

Подпись: _____

Задание принял: _____

Дата: _____

ВВЕДЕНИЕ

Работа реализует принятые требования супермаркета на React, FastAPI и PostgreSQL. Основные проверяемые свойства — серверная валидация, разграничение доступа, сохранение исторических цен, атомарная оплата и постраничная работа с большим каталогом.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Проектирование и эксплуатация базы данных

1.1 Состав таблиц и связи

products — товары; receipts — чеки; receipt_items — строки; stock_movements — движения; app_users — сотрудники; sessions — сеансы. Имена SQL латинские, предметные русские наименования приведены в комментариях и документации. Две таблицы «Товары» и «Чеки» не могут корректно представить многопозиционную покупку без связующей таблицы; поэтому добавлена receipt_items с PK(receipt_id, product_id).

У сотрудника много чеков и сеансов. В чеке 1–100 строк. Товар встречается во многих чеках; одна пара товар–чек уникальна. Движение относится к товару и сотруднику, а при продаже — к чеку. UNIQUE(receipt_id, product_id) не допускает два движения одного товара одной продажи; у ручных движений receipt_id NULL, поэтому они могут повторяться.

Деньги хранятся numeric(8,2), стоимость строки numeric(14,2) вычисляется БД; итог суммируется из строк при чтении. Копирование цены и наименования в строку сохраняет исторический документ при изменении товара. Остаток в карточке и журнал движений изменяются в одной транзакции. DB CHECK не заменяет серверную проверку точности входной цены: PostgreSQL numeric может округлить значение при прямой записи, поэтому до БД Pydantic запрещает лишние десятичные знаки.

1.2 Роли и доступ

Роль	Тип	Права
market_owner	NOLOGIN	Владелец схемы, таблиц и функций
market_admin	LOGIN, член market_owner	Управление схемой и начальными данными; не суперпользователь кластера
market_app	LOGIN	Чтение нужных таблиц, вставка карточек/чеков/движений/сеансов, обновление разрешённых столбцов, удаление сеансов

Роль	Тип	Права
market_reader	NOLOGIN	Групповые права чтения товаров, чеков, строк и движений без хешей паролей

Администратор может создать персональный LOGIN и включить его в market_reader; пароль задаётся через \password, а не хранится в SQL. PUBLIC лишается CONNECT к БД и CREATE в public. Предоставление новых таблиц не делает их автоматически доступными market_app: миграция должна явно выдать права. Приложение не работает под postgres или владельцем схемы.

Прикладные роли manager/cashier/auditor — значения app_users.role, а не отдельные PostgreSQL-соединения. Разграничение API проверяется на сервере; технические права market_app дополнительно ограничивают ущерб от ошибки. Прямой административный SQL остаётся доверенной операцией.

1.3 Индексы для объектов

Индекс	Запрос	Обоснование
PK products(id)	Карточка и соединения	Точечный доступ и стабильный ключ
UNIQUE products(barcode)	Полный штрихкод	B-tree, уникальность и быстрый equality
products_active_id_idx	Активный каталог id > cursor ORDER BY id	Частичный индекс исключает архив
products_category_id_idx	category=... AND active AND id>cursor	Фильтр и порядок в одном индексе
products_name_trgm_idx	lower(name) LIKE '%строка%' AND active	GIN pg_trgm для поиска внутри названия
receipts_cashier_id_idx	Свои чеки по ID DESC	Составной B-tree без сканирования всей истории
receipts_created_idx	История по времени для отчётного SQL	Период и устойчивый порядок
PK receipt_items(receipt_id,product_id)	Состав чека	Один диапазон индексного поиска
receipt_items_product_idx	Продажи товара	Индекс внешнего ключа и аналитика товара
stock_movements_product_time_idx	История движения товара	Поиск по товару и времени

Индекс	Запрос	Обоснование
sessions_expiry_idx	Очистка истёкших сессий	Ограничивает поиск удаляемых записей

Поиск от трёх символов снижает стоимость малоизбирательных шаблонов. Для редкой подстроки планировщик может выбрать GIN/Bitmap, для широкого запроса с небольшим LIMIT — B-tree по ID, а для большого покрытия — последовательное чтение. Наличие индекса не обязывает планировщик использовать его.

Коллация market.unicode_case создаётся через ICU с locale und. Запрос и индекс используют одинаковое выражение lower(name COLLATE unicode_case). Это обеспечивает поиск русских названий без учёта регистра даже при системной локали C. PostgreSQL должен быть собран с ICU, как выбранный официальный образ; смешанный регистр проверяется интеграционным тестом.

Пагинация id > cursor, максимум 100 записей; полный COUNT(*) не выполняется на каждую страницу. В памяти браузера хранится текущая страница. OFFSET для глубокого обхода не применяется. При 100 тысячах товаров партиционирование не обязательно: сначала измеряются EXPLAIN (ANALYZE, BUFFERS), размер таблиц, частота записи и latency API.

1.3.1 Оценка ёмкости

При условных 200–400 байтах на строку товара 100 тысяч товаров дают 20–40 МБ полезных строк без точного учёта TOAST и свободного места; индексы увеличивают объём. 100 тысяч чеков × 5 строк — 500 тысяч строк продаж и столько же движений. При 150–250 байтах строки это десятки/сотни мегабайт плюс индексы и WAL. Это инженерная оценка порядка, не замер. Для VM резервируется 40 ГБ, поскольку ОС, образы, резервные копии и временные файлы могут занимать больше собственно БД.

seed_large.sql создаёт 100 тысяч дополнительных товаров и 100 тысяч черновиков по 5 строк, фиксирует фактическое количество и

pg_total_relation_size. Набор синтетический; его нельзя загружать в рабочую базу. Он проверяет объём и планы чтения, а конкурентные оплаты проверяются отдельным API-тестом. После генерации выполняется ANALYZE.

1.4 Транзакции и ограничения

Оплата начинает транзакцию, блокирует чек FOR UPDATE и проверяет состояние. При paid возвращает существующий результат. Затем блокирует товары в порядке ID, проверяет активность и остаток, создаёт отрицательные движения, уменьшает stock и завершает чек. Возникший 409 откатывает транзакцию. Это предотвращает потерянные обновления и перепродажу последнего экземпляра. Блокировки PostgreSQL.

Триггер запрещает изменение завершённого чека и оплату пустого. Триггер вставки строки блокирует родительский чек, разрешает только draft и максимум 100 строк. Права market_app запрещают изменение/удаление строк чека. Состояние paid эквивалентно наличию paid_at. Смена цены не меняет unit_price в старых строках.

1.5 Резервное копирование и восстановление

Команды из 3lab для запущенного Compose:

```
mkdir -p backups
docker compose exec -T db pg_dump -U postgres -d supermarket -Fc > backups/supermarket.dump
docker compose exec -T db pg_restore --list < backups/supermarket.dump
docker compose exec -T db createdb -U postgres supermarket_restore
docker compose exec -T db pg_restore -U postgres -d supermarket_restore --exit-on-error <
backups/supermarket.dump
docker compose exec -T db psql -U postgres -d supermarket_restore -c 'SELECT count(*) FROM
market.products;'
```

Это восстановление в новую БД того же кластера, где роли уже существуют. При восстановлении в другой кластер сначала восстановить/создать роли; pg_dump не включает глобальные роли. Для них отдельная защищённая копия pg_dumpall --roles-only, содержащая чувствительные сведения. Доступ к новой базе ограничить теми же CONNECT/pg_hba-правилами; восстановление не означает автоматического переключения API.

Ежедневная копия и периодическая проба восстановления обеспечивают проверяемость целевых RPO/RTO; без фактического запуска расписания и восстановления достижение этих целей не утверждается. Копии хранить вне тома основной БД; удаление старых копий — отдельная явно определённая политика.

2 Нагрузочное тестирование и верификация базы данных

2.1 Нагрузочный набор и измерения

В отдельной _test или _bench БД после начальной загрузки выполнить db/seed_large.sql, затем db/explain.sql. Записать actual time, buffers и выбранные индексы. Для полного HTTP-профиля 20 конкурентных пользователей измерять p50/p95 и долю ошибок отдельно от SQL. Один EXPLAIN не подтверждает NFR02–03. Нельзя измерять все 100 тысяч объектов в браузере — это нарушает сам контракт пагинации.

2.2 Автоматический отдельный тестовый стенд

Для изоляции интеграционного тестирования от рабочей среды развертывается автономный тестовый контур с отдельной базой данных supermarket_test. Тестовый сценарий запускается в изолированной сети без публикации портов наружу, что исключает конфликт адресов и сетевое вмешательство. По завершении выполнения тестов тестовые контейнеры и тома гарантированно демонтируются обработчиком сигналов завершения. Рабочие тома с производственными данными при этом не затрагиваются. Фикстуры тестов предварительно верифицируют подключение сервисных учетных записей к выделенной тестовой схеме.

ЗАКЛЮЧЕНИЕ

Реализованы база данных, модульный FastAPI API и компонентный React-интерфейс супермаркета. Серверная валидация, роли и транзакции отделены от HTTP-маршрутов; вход, каталог, кассовая панель и история оформлены отдельными компонентами. Большой набор данных, планы SQL и восстановление копии проверены. Фактические результаты повторных тестов и границы контейнерной интеграции приведены в общем протоколе проверки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 3 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 12.09.2026).

1. FastAPI. Documentation. URL: https://fastapi.tiangolo.com/ (дата обращения: 12.09.2026).

4. PostgreSQL 17. Documentation. URL: https://www.postgresql.org/docs/17/ (дата обращения: 12.09.2026).

5. React. Documentation. URL: https://react.dev/ (дата обращения: 12.09.2026).